

Compte rendu de TP

TP PHP 5 & 6 — Architecture MVC

Étudiant Délio Duchesne-Brajon — Groupe 1
Bloc Bloc 2 — Développement
Outils PHP · MySQL · XAMPP · Visual Studio Code
Date 1^{er} octobre 2026

SOMMAIRE

1. Objectif du TP	2
2. Exercice 2 — Construction des chemins.....	2
3. Exercice 3 — Chemins portables	3
4. Exercice 4 — Injection de HTML dans la vue	4
5. Exercice 5 — Échappement des données.....	5
5.1. Affichage HTML avec htmlspecialchars	5
5.2. Liens avec rawurlencode	6
6. Exercice 7 — Vues réduites à leur contenu	8
7. Exercice 8 — Vue générique.....	10
7.1. En-tête et menu	10
7.2. Pied de page	11
8. Exercice 9 — Vue de confirmation.....	12
9. Conclusion.....	13

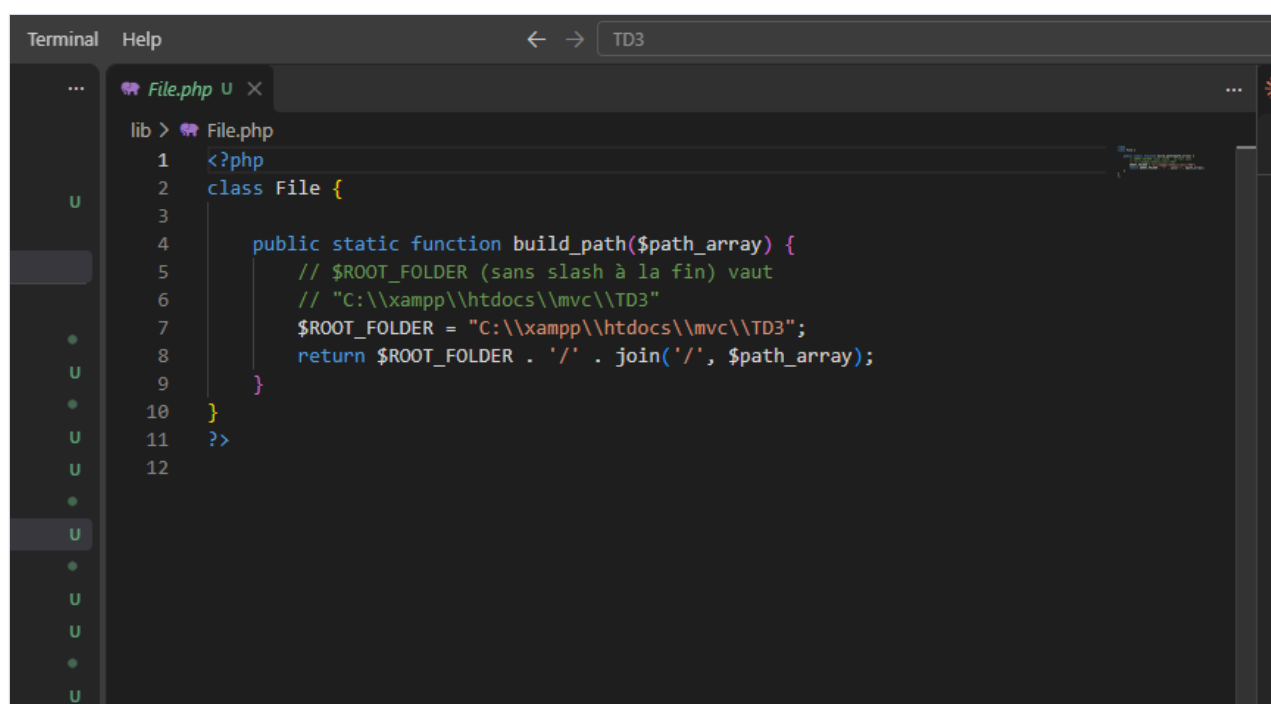
1. Objectif du TP

Poursuivre l'application MVC de gestion de voitures : rendre les chemins d'inclusion indépendants du poste, sécuriser l'affichage des données saisies par l'utilisateur, puis regrouper l'en-tête, le menu et le pied de page dans une vue générique.

Le projet est exécuté en local sous XAMPP, dans `htdocs/mvc/TD3`.

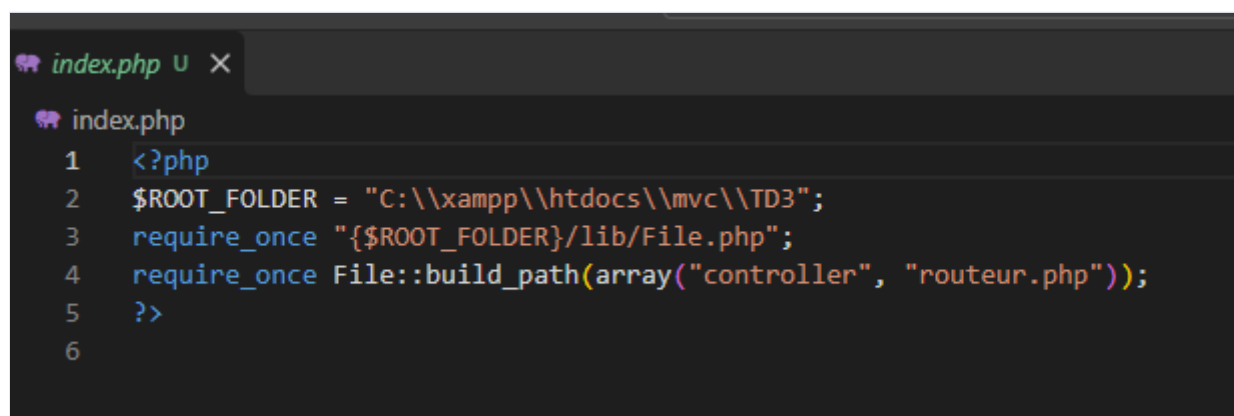
2. Exercice 2 — Construction des chemins

Une classe `File` est créée dans le dossier `lib`. Sa méthode statique `build_path()` reçoit un tableau de dossiers et renvoie le chemin complet à partir de la racine du site.



```
lib > File.php
1  <?php
2  class File {
3
4      public static function build_path($path_array) {
5          // $ROOT_FOLDER (sans slash à la fin) vaut
6          // "C:\\xampp\\htdocs\\mvc\\TD3"
7          $ROOT_FOLDER = "C:\\xampp\\htdocs\\mvc\\TD3";
8          return $ROOT_FOLDER . '/' . join('/', $path_array);
9      }
10 }
11 ?>
12
```

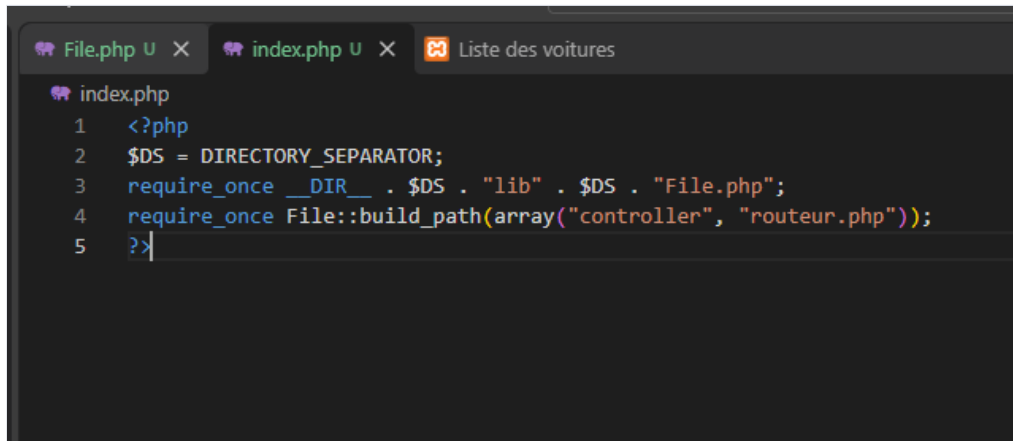
Le fichier `index.php`, placé à la racine, devient le point d'entrée : il charge `File.php` puis appelle le routeur au moyen de `build_path()`.



```
index.php
1  <?php
2  $ROOT_FOLDER = "C:\\xampp\\htdocs\\mvc\\TD3";
3  require_once "{$ROOT_FOLDER}/lib/File.php";
4  require_once File::build_path(array("controller", "routeur.php"));
5  ?>
6
```

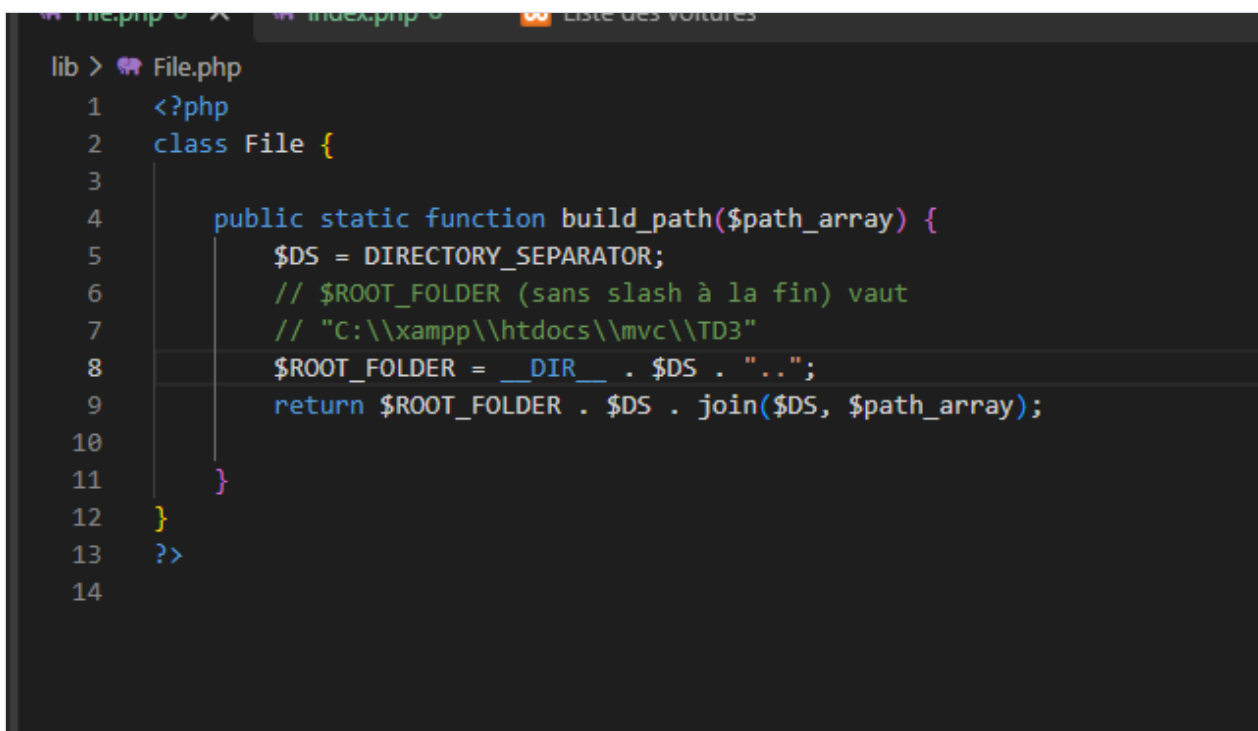
3. Exercice 3 — Chemins portables

Le chemin de la racine n'est plus écrit en dur. Le séparateur est remplacé par la constante DIRECTORY_SEPARATOR et la racine est déduite de __DIR__, ce qui rend le site utilisable sur un autre poste ou un autre système.



```
index.php
1  <?php
2  $DS = DIRECTORY_SEPARATOR;
3  require_once __DIR__ . $DS . "lib" . $DS . "File.php";
4  require_once File::build_path(array("controller", "routeur.php"));
5  ?>
```

La même modification est appliquée dans build_path(), qui remonte d'un niveau depuis le dossier lib.



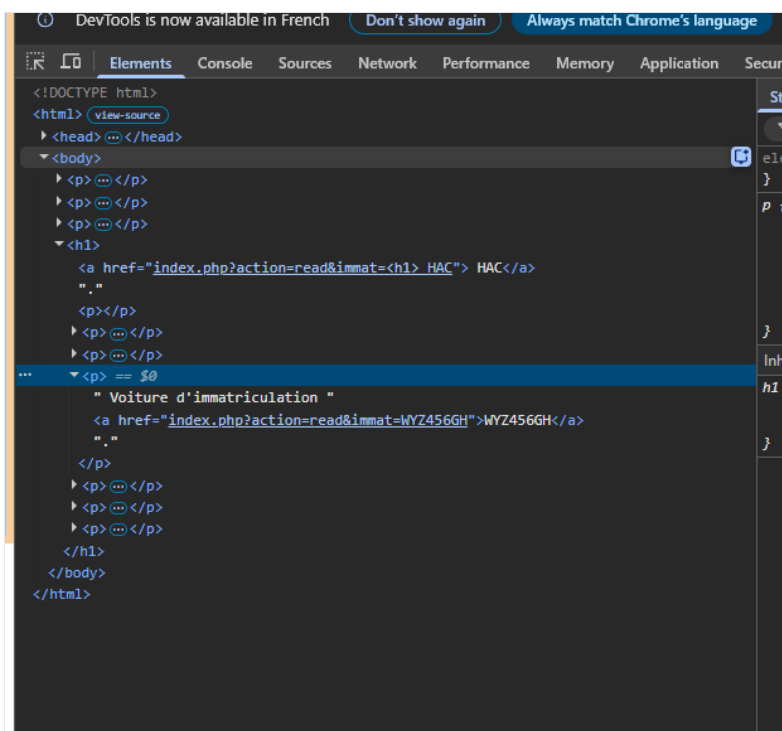
```
lib > File.php
1  <?php
2  class File {
3
4      public static function build_path($path_array) {
5          $DS = DIRECTORY_SEPARATOR;
6          // $ROOT_FOLDER (sans slash à la fin) vaut
7          // "C:\xampp\htdocs\mvc\TD3"
8          $ROOT_FOLDER = __DIR__ . $DS . "..";
9          return $ROOT_FOLDER . $DS . join($DS, $path_array);
10     }
11 }
12
13 ?>
14
```

4. Exercice 4 — Injection de HTML dans la vue

Une voiture est enregistrée avec l'immatriculation `<h1> HAC`. La liste affiche cette valeur sans la traiter : le navigateur l'interprète comme une balise et tout le reste de la page passe en titre.



L'inspecteur du navigateur le confirme : une balise `<h1>` a été insérée dans le document et englobe les paragraphes suivants.



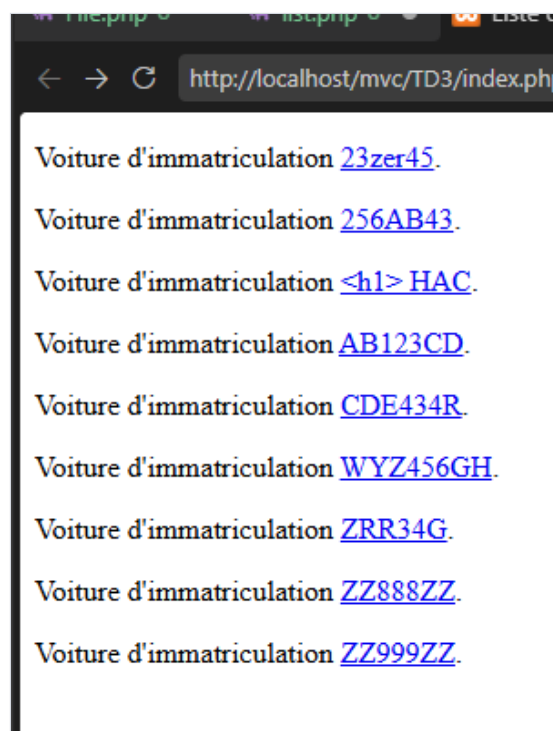
5. Exercice 5 — Échappement des données

5.1. Affichage HTML avec htmlspecialchars

Dans la vue `list.php`, chaque attribut est passé à `htmlspecialchars()` avant d'être affiché.

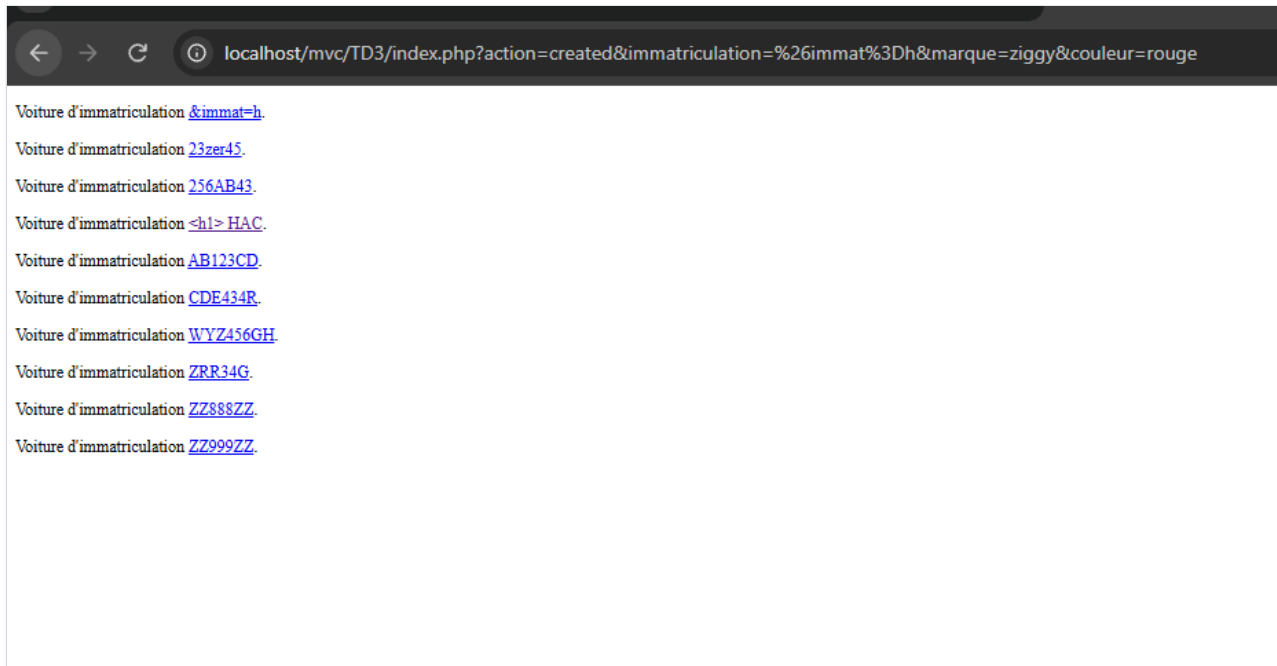
```
view > voiture > 🐞 list.php
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="UTF-8">
5  <title>Liste des voitures</title>
6  </head>
7  <body>
8  <?php
9  foreach ($tab_v as $v) {
10   $vImmat = htmlspecialchars($v->getImmatriculation());
11   $vMarque = htmlspecialchars($v->getMarque());
12   $vCouleur = htmlspecialchars($v->getCouleur());
13   echo '<p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmat . '>';
14   }
15
16  ?>
17  </body>
18  </html>
19
```

Les caractères spéciaux sont convertis en entités : l'immatriculation s'affiche telle qu'elle a été saisie et la page retrouve sa mise en forme.



5.2. Liens avec rawurlencode

Le même problème se pose dans les URL. Une voiture d'immatriculation &immat=h est créée puis apparaît dans la liste.



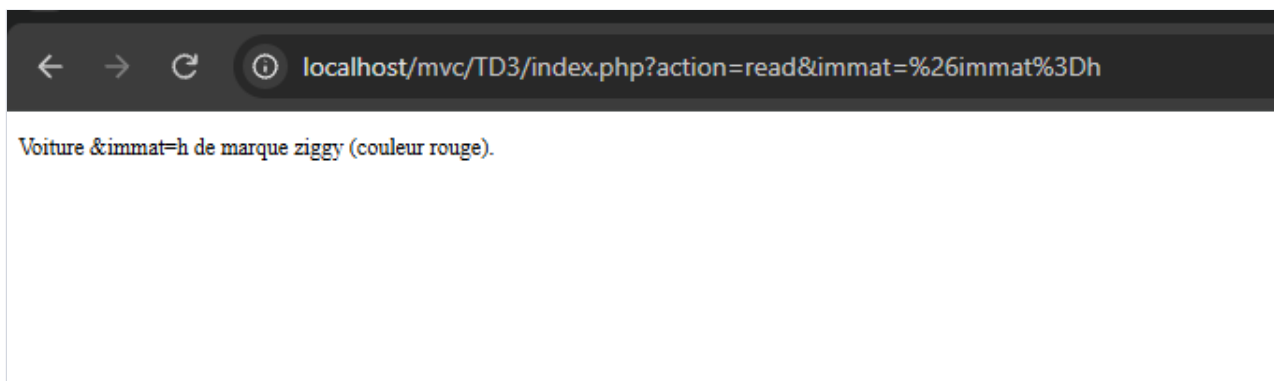
Son lien de détail ne fonctionne pas : le caractère & est lu comme un séparateur de paramètres, l'immatriculation transmise est vide et la page d'erreur s'affiche.



Une variable `$vImmatURL`, obtenue avec `rawurlencode()`, est utilisée dans l'attribut `href` du lien.

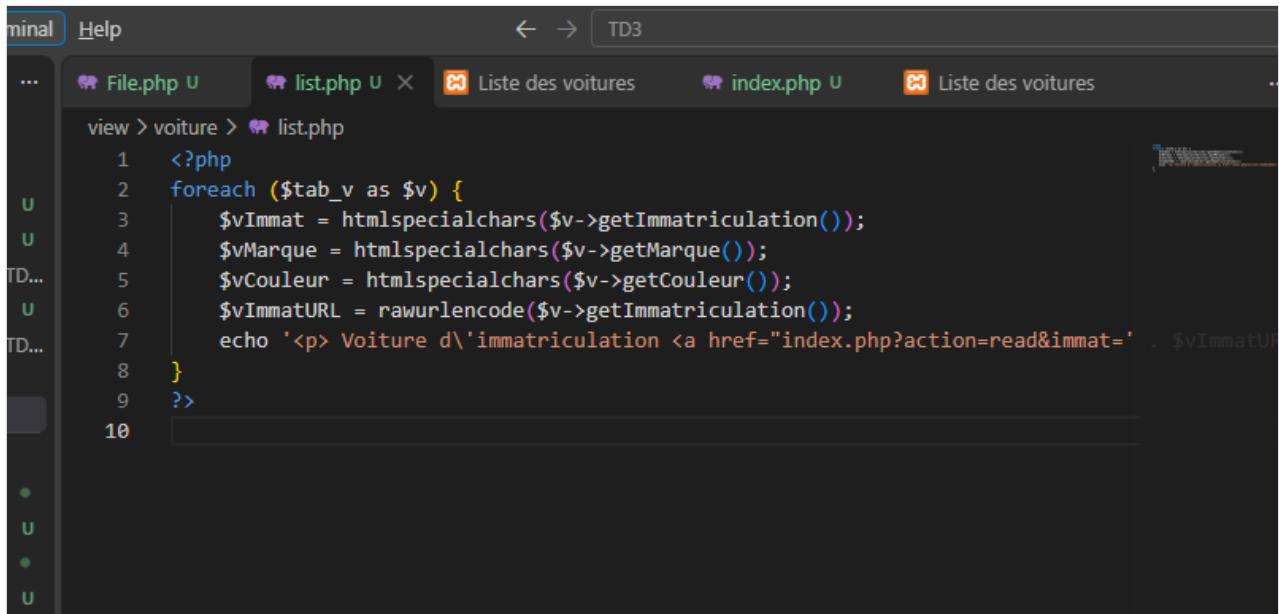
```
view > voiture > list.php
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="UTF-8">
5      <title>Liste des voitures</title>
6    </head>
7    <body>
8      <?php
9        foreach ($tab_v as $v) {
10          $vImmat = htmlspecialchars($v->getImmatriculation());
11          $vMarque = htmlspecialchars($v->getMarque());
12          $vCouleur = htmlspecialchars($v->getCouleur());
13          $vImmatURL = rawurlencode($v->getImmatriculation());
14          echo '<p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmatURL . '>';
15        }
16
17      ?>
18    </body>
19  </html>
20
```

L'immatriculation est alors encodée dans l'URL (`%26immat%3Dh`) et le détail de la voiture s'affiche correctement.



6. Exercice 7 — Vues réduites à leur contenu

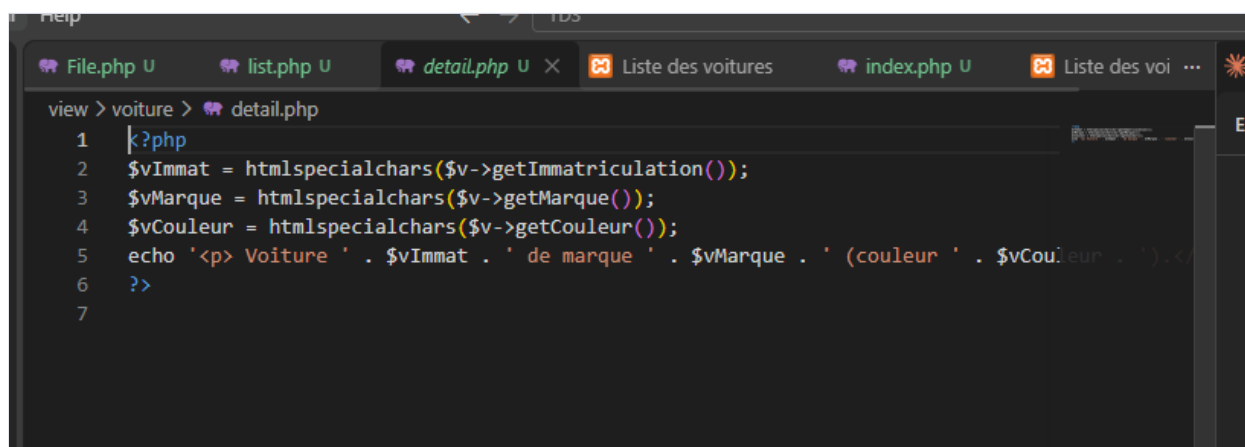
Les vues ne contiennent plus la structure complète de la page HTML, seulement leur contenu propre. La vue `list.php` se limite à la boucle d'affichage.



The screenshot shows a code editor with a dark theme. The top bar includes a 'Terminal' tab and a 'Help' button. The editor has several tabs: 'File.php U', 'list.php U', 'Liste des voitures', 'index.php U', and another 'Liste des voitures'. The active tab is 'list.php U', showing the following PHP code:

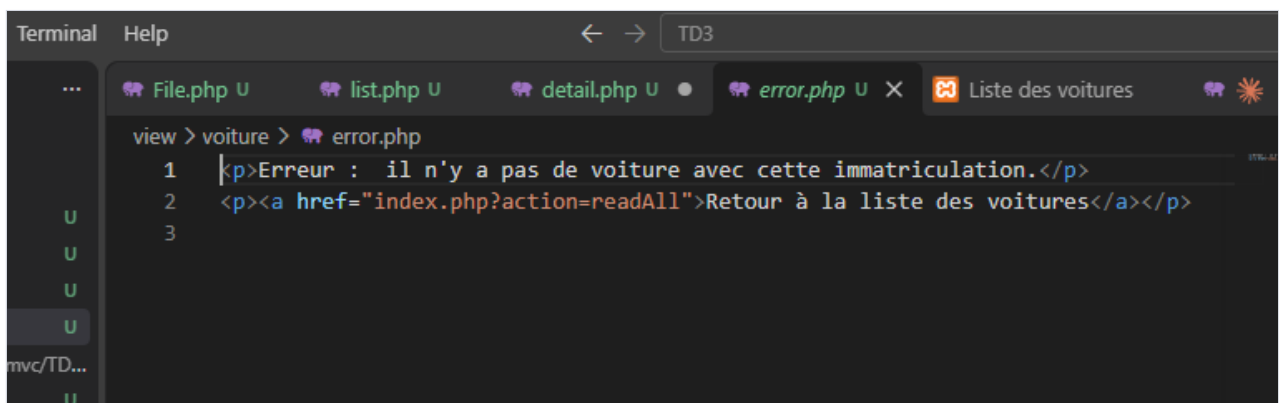
```
view > voiture > list.php
1  <?php
2  foreach ($tab_v as $v) {
3      $vImmat = htmlspecialchars($v->getImmatriculation());
4      $vMarque = htmlspecialchars($v->getMarque());
5      $vCouleur = htmlspecialchars($v->getCouleur());
6      $vImmatURL = rawurlencode($v->getImmatriculation());
7      echo '<p> Voiture d\'immatriculation <a href="index.php?action=read&immat=' . $vImmatURL
8  }
9  ?>
10
```

Les vues `detail.php` et `error.php` sont simplifiées de la même manière.



The screenshot shows a code editor with a dark theme. The top bar includes a 'Terminal' tab and a 'Help' button. The editor has several tabs: 'File.php U', 'list.php U', 'detail.php U', 'Liste des voitures', 'index.php U', and another 'Liste des voi...'. The active tab is 'detail.php U', showing the following PHP code:

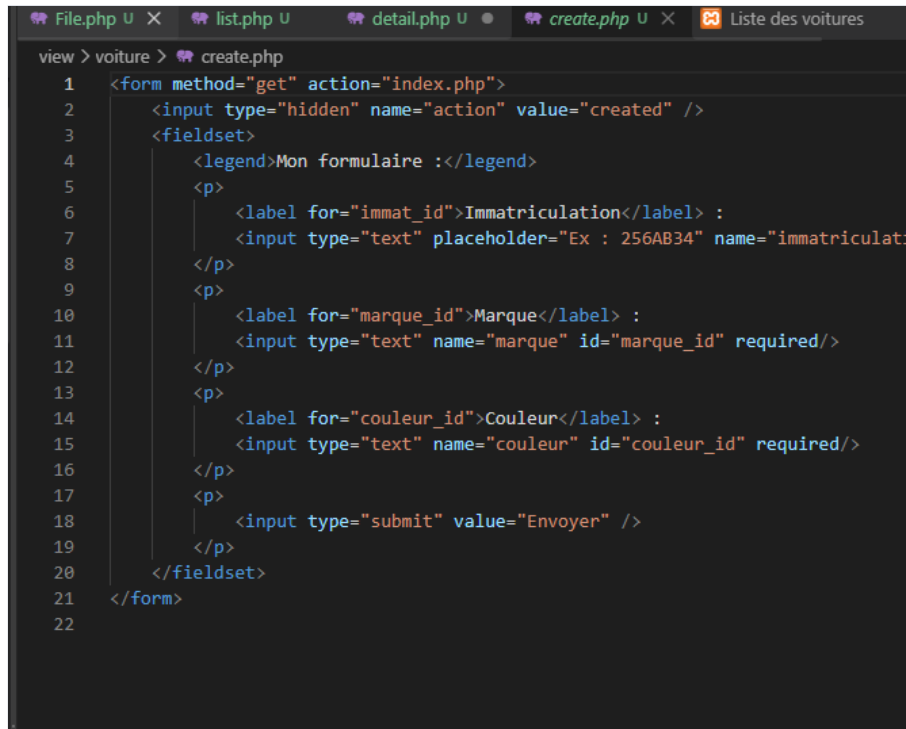
```
view > voiture > detail.php
1  <?php
2  $vImmat = htmlspecialchars($v->getImmatriculation());
3  $vMarque = htmlspecialchars($v->getMarque());
4  $vCouleur = htmlspecialchars($v->getCouleur());
5  echo '<p> Voiture ' . $vImmat . ' de marque ' . $vMarque . ' (couleur ' . $vCouleur . ')</p>';
6  ?>
7
```



The screenshot shows a code editor with a dark theme. The top bar includes a 'Terminal' tab and a 'Help' button. The editor has several tabs: 'File.php U', 'list.php U', 'detail.php U', 'error.php U', and 'Liste des voitures'. The active tab is 'error.php U', showing the following PHP code:

```
view > voiture > error.php
1  <p>Erreur : il n'y a pas de voiture avec cette immatriculation.</p>
2  <p><a href="index.php?action=readAll">Retour à la liste des voitures</a></p>
3
```

Le formulaire de create.php envoie désormais ses données en get vers index.php, l'action created étant transmise par un champ caché.

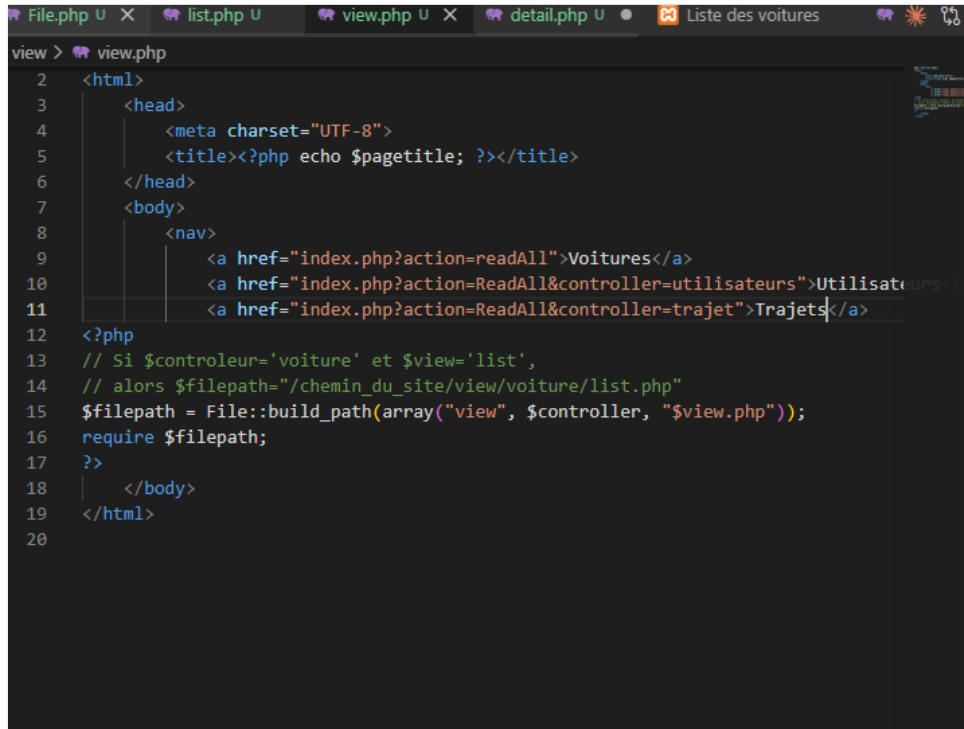


```
1 <form method="get" action="index.php">
2   <input type="hidden" name="action" value="created" />
3   <fieldset>
4     <legend>Mon formulaire :</legend>
5     <p>
6       <label for="immat_id">Immatriculation</label> :
7       <input type="text" placeholder="Ex : 256AB34" name="immatriculation" />
8     </p>
9     <p>
10      <label for="marque_id">Marque</label> :
11      <input type="text" name="marque" id="marque_id" required/>
12    </p>
13    <p>
14      <label for="couleur_id">Couleur</label> :
15      <input type="text" name="couleur" id="couleur_id" required/>
16    </p>
17    <p>
18      <input type="submit" value="Envoyer" />
19    </p>
20  </fieldset>
21 </form>
22
```

7. Exercice 8 — Vue générique

7.1. En-tête et menu

Le fichier `view/view.php` porte la structure commune à toutes les pages : le titre, lu dans `$pagetitle`, et un menu vers les voitures, les utilisateurs et les trajets. Le contenu est inclus à partir des variables `$controller` et `$view`, grâce à `build_path()`.



```
1 <?php
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title><?php echo $pagetitle; ?></title>
6   </head>
7   <body>
8     <nav>
9       <a href="index.php?action=readAll">Voitures</a>
10      <a href="index.php?action=ReadAll&controller=utilisateurs">Utilisateurs</a>
11      <a href="index.php?action=ReadAll&controller=trajet">Trajets</a>
12    </nav>
13    <?php
14      // Si $controller='voiture' et $view='list',
15      // alors $filepath="/chemin_du_site/view/voiture/list.php"
16      $filepath = File::build_path(array("view", $controller, "$view.php"));
17      require $filepath;
18    </body>
19 </html>
20
```

7.2. Pied de page

Un pied de page est ajouté à la vue générique.

```
File.php U list.php U view.php U X detail.php U Liste des voitures
view > view.php
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title><?php echo $pagetitle; ?></title>
6   </head>
7   <body>
8     <nav>
9       <a href="index.php?action=readAll">Voitures</a>
10      <a href="index.php?action=ReadAll&controller=utilisateurs">Utilisateurs</a>
11      <a href="index.php?action=ReadAll&controller=trajet">Trajets</a>
12    </nav>
13    <?php
14    // Si $controleur='voiture' et $view='list',
15    // alors $filepath="/chemin_du_site/view/voiture/list.php"
16    $filepath = File::build_path(array("view", $controller, "$view.php"));
17    require $filepath;
18  </body>
19  <footer>
20    <p style="border: 1px solid black;text-align:right;padding-right:1em;">
21      Site de covoiturage de Délío
22    </p>
23  </footer>
24 </html>
25
```

Le menu et le pied de page apparaissent maintenant sur la liste des voitures, qui affiche aussi la marque et la couleur.

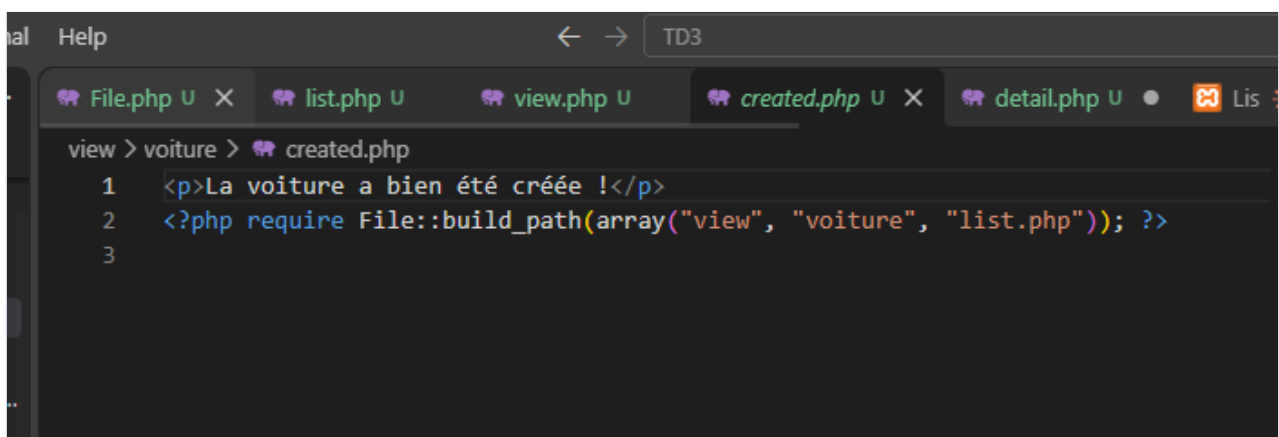
don
Voitures Utilisateurs Trajets
Voiture d'immatriculation 23zer45 de marque ziggy (couleur rouge).
Voiture d'immatriculation 23zer45 de marque nissan (couleur pomme).
Voiture d'immatriculation 256AB43 de marque ezzz (couleur dfssdf).
Voiture d'immatriculation <h1>HAC de marque guez (couleur bleu).
Voiture d'immatriculation AB123CD de marque peugeot (couleur vert).
Voiture d'immatriculation CDE434R de marque nissan (couleur orange).
Voiture d'immatriculation WYZ456GH de marque ferrari (couleur rouge).
Voiture d'immatriculation ZRR34G de marque hyundai (couleur blanc).
Voiture d'immatriculation ZZ888ZZ de marque Renault (couleur bleu).
Voiture d'immatriculation ZZ999ZZ de marque Renault (couleur bleu).
Site de covoiturage de Délío

8. Exercice 9 — Vue de confirmation

La liste est affichée à travers la vue générique.

Voitures Utilisateurs Trajets
Voiture d'immatriculation &immat=h de marque ziggy (couleur rouge).
Voiture d'immatriculation 23zer45 de marque nissan (couleur pomme).
Voiture d'immatriculation 256AB43 de marque ezrz (couleur dfssdf).
Voiture d'immatriculation <h1>HAC de marque guez (couleur bleu).
Voiture d'immatriculation AB123CD de marque peugeot (couleur vert).
Voiture d'immatriculation CDE434R de marque nissan (couleur orange).
Voiture d'immatriculation WYZ456GH de marque ferrari (couleur rouge).
Voiture d'immatriculation ZRR34G de marque hyundai (couleur blanc).
Voiture d'immatriculation ZZ888ZZ de marque Renault (couleur bleu).
Voiture d'immatriculation ZZ999ZZ de marque Renault (couleur bleu).
Site de covoiturage de Délio

Une vue `created.php` affiche un message de confirmation puis réutilise `list.php`, sans dupliquer le code d'affichage.



```
view > voiture > created.php
1 <p>La voiture a bien été créée !</p>
2 <?php require File::build_path(array("view", "voiture", "list.php")); ?>
3
```

Après l'envoi du formulaire, le message s'affiche au-dessus de la liste, qui contient la nouvelle voiture.

[Voitures](#) [Utilisateurs](#) [Trajets](#)

La voiture a bien été créée !

Voiture d'immatriculation [&immat=h](#) de marque ziggy (couleur rouge).

Voiture d'immatriculation [23zer45](#) de marque nissan (couleur pomme).

Voiture d'immatriculation [256AB43](#) de marque ezrz (couleur dfssdf).

Voiture d'immatriculation [45az65](#) de marque mercedes (couleur jaune).

Voiture d'immatriculation [<h1>HAC](#) de marque guez (couleur bleu).

Voiture d'immatriculation [AB123CD](#) de marque peugeot (couleur vert).

Voiture d'immatriculation [CDE434R](#) de marque nissan (couleur orange).

Voiture d'immatriculation [WYZ456GH](#) de marque ferrari (couleur rouge).

Voiture d'immatriculation [ZRR34G](#) de marque hyundai (couleur blanc).

Voiture d'immatriculation [ZZ888ZZ](#) de marque Renault (couleur bleu).

Voiture d'immatriculation [ZZ999ZZ](#) de marque Renault (couleur bleu).

Site de covoiturage de Délio

9. Conclusion

Ce TP rend l'application plus solide sur trois points : les chemins ne dépendent plus du poste, les données saisies sont échappées avant d'être affichées ou placées dans une URL, et la structure HTML est écrite une seule fois dans la vue générique.

L'échappement est le point le plus important : sans `htmlspecialchars()` et `rawurlencode()`, une simple saisie suffit à casser la page ou un lien.